

Documentación Técnica — Módulo de Gestión de Bandas

Proyecto: Music Hub

Rama: `feature/gestion-bandas` (fusionada con `feature/instrumentos-sistema-personalizados`)

Fecha: Mayo 2026

Framework: Symfony 8.0 · PHP 8.4 · Doctrine ORM · MySQL · Twig · Bootstrap 5

Índice

1. [Resumen de la funcionalidad](#)
 2. [Tecnologías utilizadas](#)
 3. [Cambios en la base de datos](#)
 4. [Entidades modificadas o creadas](#)
 5. [Refactorización del sistema de instrumentos](#)
 6. [Sistema de gestión de bandas](#)
 7. [Ciclo de vida de la membresía](#)
 8. [Sistema de invitaciones de banda a músico](#)
 9. [Sistema de administradores](#)
 10. [Integración con Google Places API](#)
 11. [Rutas implementadas](#)
 12. [Controladores](#)
 13. [Formularios y validaciones](#)
 14. [Vistas Twig](#)
 15. [Navbar dinámico](#)
 16. [Seguridad y protección CSRF](#)
 17. [Migraciones de base de datos](#)
-

1. Resumen de la funcionalidad

Esta rama implementa el **módulo completo de gestión de bandas** para la aplicación Music Hub, una plataforma que conecta músicos y agrupaciones musicales.

Antes de esta rama, el CRUD básico de `Banda` existía pero era solo un esqueleto sin lógica de membresía ni flujo real de usuarios. En esta rama se ha construido desde cero:

- El sistema de **solicitudes de unión** (un músico pide entrar a una banda).
- El sistema de **invitaciones inversas** (una banda invita a un músico concreto).
- La figura del **administrador de banda**, con capacidad de aceptar/rechazar solicitudes, gestionar miembros, otorgar o revocar permisos de admin, expulsar miembros y editar sus roles.
- La posibilidad de que un músico **salga voluntariamente** de una banda.
- El **dropdown "Gestiona tu banda"** en la barra de navegación, que muestra dinámicamente las bandas a las que pertenece el usuario.

- La integración con la **API de Google Places** para autocompletar la ubicación de la banda y guardar sus coordenadas geográficas.
- La **refactorización del sistema de instrumentos**, separando los instrumentos del sistema (predefinidos) de los instrumentos personalizados creados por el propio músico.
- Validaciones de formulario completas en el formulario de banda.

2. Tecnologías utilizadas

Capa	Tecnología
Backend	PHP 8.4, Symfony 8.0
ORM	Doctrine ORM, Doctrine Migrations
Frontend (plantillas)	Twig 3, Bootstrap 5 (tema Bootswatch Vapor)
JavaScript (componentes)	Stimulus (via AssetMapper de Symfony)
API externa	Google Places API (Autocomplete)
Base de datos	MySQL (Docker)
Entorno de desarrollo	Docker Desktop en Windows 11
Control de versiones	Git

3. Cambios en la base de datos

Se han creado **tres migraciones** nuevas en esta rama. Las migraciones en Doctrine permiten evolucionar el esquema de la base de datos de forma controlada y reproducible; cada migración tiene un método `up()` (aplica el cambio) y `down()` (lo deshace).

Migración 1 — `Version20260524120000`

(proveniente de la rama `feature/instrumentos-sistema-personalizados`)

Refactoriza la tabla de instrumentos, separando los que son del sistema de los personalizados:

```
-- Crea tabla de instrumentos del sistema (predefinidos por administradores)
CREATE TABLE instrumento_sistema (id INT, nombre VARCHAR(100), ...);

-- Crea tabla de instrumentos personalizados (creados por el músico)
CREATE TABLE instrumento_personalizado (id INT, nombre VARCHAR(100), musico_id INT, ...);

-- Tablas de relación ManyToMany músico ↔ instrumento
CREATE TABLE musico_instrumento_sistema (...);
CREATE TABLE musico_instrumento_personalizado (...);

-- Elimina la tabla antigua que mezclaba ambos conceptos
DROP TABLE instrumento_musico;
```

Migración 2 — Version20260527120000

(núcleo del módulo de gestión de bandas)

Añade los campos necesarios para la gestión de membresía:

```
-- Añade nombre a la banda (antes no tenía campo nombre explícito)
ALTER TABLE banda ADD nombre VARCHAR(100) NOT NULL DEFAULT '';
UPDATE banda SET nombre = CONCAT('Banda ', id) WHERE nombre = '';

-- Añade el estado de la solicitud/membresía
-- Valores posibles: 'pendiente', 'aceptado', 'rechazado', 'invitado'
ALTER TABLE miembro_banda ADD estado VARCHAR(20) NOT NULL DEFAULT 'aceptado';

-- Hace que rol_banda sea opcional (antes era obligatorio)
ALTER TABLE miembro_banda MODIFY rol_banda VARCHAR(100) NULL;

-- Añade si el miembro es administrador de la banda
ALTER TABLE miembro_banda ADD es_administrador TINYINT(1) NOT NULL DEFAULT 0;
```

Migración 3 — Version20260527130000

(soporte para coordenadas geográficas)

```
-- Coordenadas para integración con Google Maps / Places
ALTER TABLE banda ADD latitud DOUBLE PRECISION DEFAULT NULL;
ALTER TABLE banda ADD longitud DOUBLE PRECISION DEFAULT NULL;
```

4. Entidades modificadas o creadas

Banda (modificada)

Archivo: src/Entity/Banda.php

La entidad banda se ha ampliado con tres nuevos campos:

```
#[ORM\Column(length: 100)]
private ?string $nombre = null;           // Nombre de la banda

#[ORM\Column(nullable: true)]
private ?float $latitud = null;           // Latitud (Google Places)

#[ORM\Column(nullable: true)]
private ?float $longitud = null;          // Longitud (Google Places)
```

La relación con **MiembroBanda** ya existía como **OneToMany**. Se mantiene y se aprovecha para filtrar miembros por estado directamente en las plantillas Twig.

MiembroBanda (modificada)

Archivo: src/Entity/MiembroBanda.php

Esta entidad es la **tabla intermedia** que une a un **Musico** con una **Banda**. Es el corazón del sistema de membresía. Se le han añadido dos campos fundamentales:

```
// El rol que ocupa en la banda (ej: "Guitarra, Bajo", "Fundador")
// Es nullable porque al crear la banda el fundador no tiene que elegir
instrumento
#[ORM\Column(length: 100, nullable: true)]
private ?string $rol_banda = null;

// Estado de la relación músico-banda
// 'pendiente' → el músico ha pedido unirse, espera aprobación
// 'aceptado' → miembro activo de la banda
// 'rechazado' → su solicitud fue denegada
// 'invitado' → la banda le ha enviado una invitación, él aún no ha respondido
#[ORM\Column(length: 20)]
private string $estado = 'pendiente';

// Si este miembro tiene permisos de administrador en la banda
#[ORM\Column]
private bool $es_administrador = false;
```

Con este diseño, una única entidad **MiembroBanda** puede representar cualquier tipo de relación entre músico y banda a lo largo de todo su ciclo de vida.

Musico (modificada)

Archivo: src/Entity/Musico.php

Se han añadido dos colecciones ManyToMany para los instrumentos refactorizados, y dos métodos auxiliares:

```
// Relaciones con instrumentos (ver sección 5)
#[ORM\ManyToMany(targetEntity: InstrumentoSistema::class)]
#[ORM\JoinTable(name: 'musico_instrumento_sistema')]
private Collection $instrumentosSistema;

#[ORM\ManyToMany(targetEntity: InstrumentoPersonalizado::class)]
#[ORM\JoinTable(name: 'musico_instrumento_personalizado')]
private Collection $instrumentosPersonalizados;

// Devuelve todos los instrumentos combinados (sistema + personalizados)
// Usado en plantillas con {{ musico.instrumentos }}
public function getInstrumentos(): Collection
{
    return new ArrayCollection(array_merge(
        $this->instrumentosSistema->toArray(),
        $this->instrumentosPersonalizados->toArray()
    ));
}

// Devuelve solo las bandas donde el músico está aceptado
// Usado en el navbar y en el perfil del músico
public function getMiembroBandasAceptadas(): Collection
{
    return $this->miembroBandas->filter(
        fn(MiembroBanda $mb) => $mb->getEstado() === 'aceptado'
    );
}
```

InstrumentoSistema y InstrumentoPersonalizado (nuevas)

Archivos: `src/Entity/InstrumentoSistema.php`, `src/Entity/InstrumentoPersonalizado.php`

Sustituyen a la antigua entidad `InstrumentoMusico`. Ver sección 5.

5. Refactorización del sistema de instrumentos

Situación anterior

Existía una única entidad `InstrumentoMusico` que mezclaba instrumentos predefinidos del sistema con instrumentos creados por los músicos. Esto causaba problemas de diseño y la tabla `instrumento_musico` fue eliminada.

Situación nueva

Se separan en dos entidades independientes:

InstrumentoSistema — Instrumentos predefinidos por los administradores de la aplicación (guitarra, bajo, batería, etc.). Cualquier músico puede seleccionarlos.

InstrumentoPersonalizado — Instrumentos creados por el propio músico en su perfil. Solo pertenecen a ese músico.

Ambos tipos se relacionan con **Musico** mediante tablas intermedias ManyToMany (`musico_instrumento_sistema`, `musico_instrumento_personalizado`). El método auxiliar `getInstrumentos()` en la entidad **Musico** combina ambas colecciones de forma transparente para las plantillas.

Por qué era necesario el merge

En el momento de desarrollar el módulo de gestión de bandas, la rama `feature/instrumentos-sistema-personalizados` ya había eliminado la tabla `instrumento_musico` pero no estaba fusionada en la rama principal. Cuando el perfil de un músico intentaba mostrar sus instrumentos desde la página de solicitudes de la banda, Doctrine intentaba consultar la tabla `instrumento_musico` que ya no existía, produciendo un error 500.

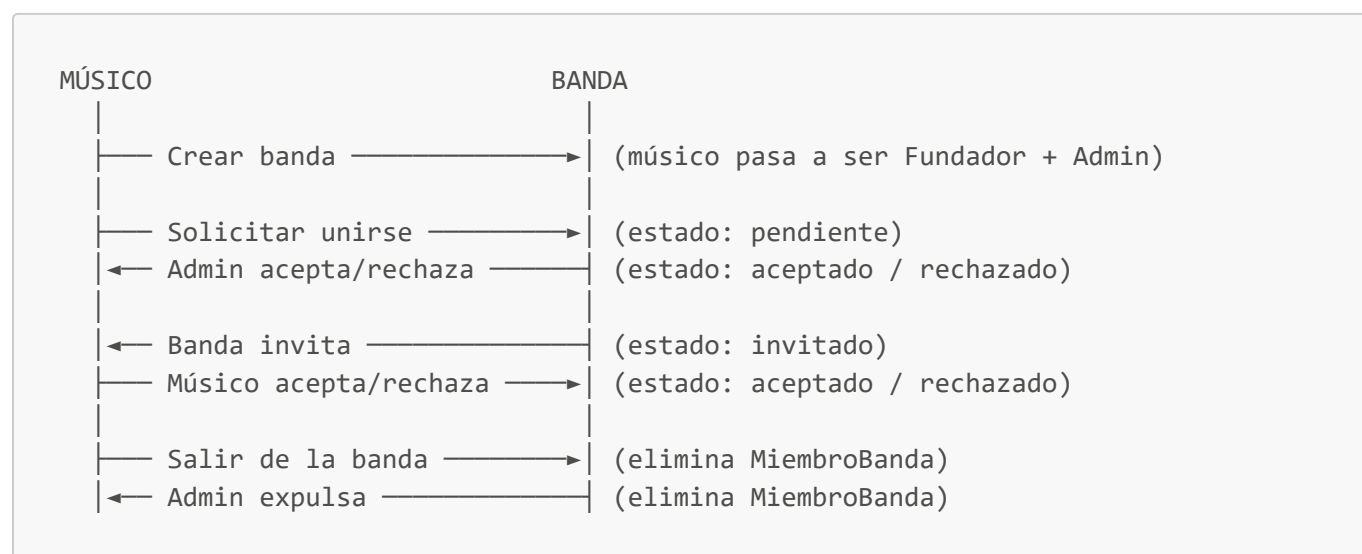
La solución fue realizar un **merge** de `feature/instrumentos-sistema-personalizados` en `feature/gestion-bandas`, resolviendo el conflicto en `Musico.php` que se produjo entre:

- La versión antigua (que referenciaba `InstrumentoMusico`)
- La versión nueva (que usa los dos ManyToMany)

Se mantuvo la versión nueva de las relaciones y se añadió el método `getMiembroBandasAceptadas()` que pertenece a esta rama.

6. Sistema de gestión de bandas

Visión general del flujo



Creación de una banda

Cuando un músico crea una banda, el sistema:

1. Persiste la entidad **Banda** con los datos del formulario.
2. Crea automáticamente un **MiembroBanda** con:
 - `estado = 'aceptado'`
 - `es_administrador = true`
 - `rol_banda = 'Fundador'`
3. Redirige al perfil de la nueva banda.

```
// En BandaController::new()
$miembro = new MiembroBanda();
$miembro->setBanda($banda);
$miembro->setMusico($musico);
$miembro->setRolBanda('Fundador');
$miembro->setEstado('aceptado');
$miembro->setEsAdministrador(true);
$entityManager->persist($miembro);
```

7. Ciclo de vida de la membresía

El campo `estado` en **MiembroBanda** puede tomar cuatro valores. A continuación se describe cada uno y cuándo se produce la transición:

pendiente

El músico ha rellenado el formulario de solicitud de unión. El estado inicial de toda solicitud iniciada por un músico. Los administradores de la banda pueden ver estas solicitudes en la página "Gestionar banda".

aceptado

Un administrador ha aceptado la solicitud (o el músico ha aceptado una invitación). El músico aparece en el perfil de la banda como miembro activo.

rechazado

Un administrador ha rechazado la solicitud (o el músico ha rechazado una invitación). El músico ve un mensaje en el perfil de la banda informándole del rechazo.

invitado

La banda ha tomado la iniciativa de invitar al músico. El músico verá la invitación pendiente en su propio perfil y podrá aceptarla o rechazarla.

Solicitud de unión — flujo detallado

1. El músico visita el perfil de una banda y pulsa **"Solicitar unirse"**.
2. Se le muestra un formulario con sus instrumentos (del sistema + personalizados) como checkboxes.
3. Al enviar, se guarda un **MiembroBanda** con `estado='pendiente'` y `rol_banda` con los instrumentos seleccionados en formato texto ("**Guitarra**, **Bajo**").

4. El administrador accede a "**Gestionar banda**" y ve la solicitud pendiente con el nombre del músico, su ubicación y los instrumentos con los que se ha presentado.
 5. El admin acepta o rechaza. En ambos casos se actualiza el campo `estado`.
-

8. Sistema de invitaciones de banda a músico

Este flujo es el inverso: la banda toma la iniciativa de contactar a un músico.

Cómo funciona

1. Un administrador de banda visita el **perfil de un músico** (desde el listado de músicos).
2. Si el músico no pertenece ya a esa banda, aparece un botón "**Invitar a [nombre de banda]**" por cada banda que administra el usuario actual.
3. Al pulsar, se crea un `MiembroBanda` con `estado='invitado'`.
4. El músico, al visitar su propio perfil, ve una sección "**Invitaciones pendientes**" con el nombre de la banda y los botones "Aceptar" / "Rechazar".
5. Si acepta, el `estado` pasa a `'aceptado'`. Si rechaza, pasa a `'rechazado'`.

Protecciones implementadas

- Solo el músico destinatario puede gestionar su propia invitación (se comprueba que `$miembro->getMusico() === $musicoActual`).
 - No se crea una nueva invitación si ya existe una relación activa (pendiente, aceptada o invitada). Solo se permite reinvitar si el estado anterior era `'rechazado'`.
-

9. Sistema de administradores

Permisos del administrador

Un usuario con `es_administrador = true` en un `MiembroBanda` puede:

- Ver y gestionar solicitudes pendientes de unión.
- Aceptar o rechazar solicitudes.
- Otorgar permisos de administrador a otros miembros.
- Revocar permisos de administrador a otros administradores.
- Expulsar miembros de la banda.
- Editar el rol/instrumentos de cualquier miembro.
- Editar los datos generales de la banda.
- Eliminar la banda.
- Invitar músicos desde sus perfiles.

Protecciones contra quedar sin admin

Dos acciones podrían dejar una banda sin ningún administrador: revocar el último admin o expulsar al último admin. Ambas situaciones están protegidas en el controlador con la misma lógica:


```
$totalAdmins = $banda->getMiembroBandas()->filter(
    fn(MiembroBanda $mb) => $mb->isEsAdministrador() && $mb->getEstado() ===
    'aceptado'
)->count();

if ($totalAdmins <= 1) {
    $this->addFlash('warning', 'No puedes quitar el último administrador de la
    banda.');
```

```
    return $this->redirectToRoute('app_banda_solicitudes', ['id' => $banda-
    >getId()]);
}
```

Esta misma lógica aplica a "Salir de la banda": si un administrador intenta salir y es el único admin, el sistema le avisa de que primero debe designar a otro administrador.

Método auxiliar privado

Para no repetir la comprobación de admin en cada acción del controlador, existe un método privado reutilizable:

```
private function esAdminDeBanda(Banda $banda, Musico $musico): bool
{
    foreach ($banda->getMiembroBandas() as $mb) {
        if ($mb->getMusico() === $musico
            && $mb->isEsAdministrador()
            && $mb->getEstado() === 'aceptado') {
            return true;
        }
    }
    return false;
}
```

Este método es llamado en todas las acciones que requieren permisos de administrador. Si el usuario no cumple la condición, se lanza una excepción 403 con `$this->createAccessDeniedException()`.

10. Integración con Google Places API

Propósito

Cuando un usuario rellena la ubicación de su banda, en lugar de escribir texto libre se activa el autocompletado de Google Places. Al seleccionar una sugerencia, el campo de texto se rellena con la dirección formateada y se guardan automáticamente las coordenadas de latitud y longitud en campos ocultos. Esto permite una ubicación estandarizada y con datos geográficos precisos.

Implementación técnica

La integración usa **Stimulus**, el framework de JavaScript ligero que viene con Symfony AssetMapper. El controlador Stimulus ([google-places-autocomplete](#)) se conecta al `div` que envuelve el campo de ubicación mediante atributos `data-*` en el HTML:

```
{# templates/banda/_form.html.twig #}
<div class="col-md-6"
    data-controller="google-places-autocomplete"
    data-google-places-autocomplete-api-key-value="{{ google_places_api_key }}">
    {{ form_widget(form.ubicacion, {'attr': {'class': 'form-control'}}) }}
    {{ form_widget(form.latitud, {'attr': {'data-google-places-autocomplete-
target': 'lat'}}) }}
    {{ form_widget(form.longitud, {'attr': {'data-google-places-autocomplete-
target': 'lng'}}) }}
</div>
```

La clave de API se inyecta desde la variable de entorno `GOOGLE_PLACES_API_KEY` a través de `config/services.yaml`, y se pasa a las plantillas con un parámetro Twig global (`google_places_api_key`).

Los campos `latitud` y `longitud` se declaran en el formulario como `HiddenType`, de modo que son invisibles para el usuario pero son enviados junto con el formulario:

```
->add('latitud', HiddenType::class, ['required' => false])
->add('longitud', HiddenType::class, ['required' => false])
```

11. Rutas implementadas

Todas las rutas del módulo de bandas están bajo el prefijo `/banda`, definidas mediante atributos PHP en `BandaController`.

Método HTTP	URL	Nombre de ruta	Descripción
GET	<code>/banda/list</code>	<code>app_banda_index</code>	Listado de todas las bandas
GET POST	<code>/banda/new</code>	<code>app_banda_new</code>	Crear nueva banda
GET	<code>/banda/{id}</code>	<code>app_banda_show</code>	Perfil público de una banda
GET POST	<code>/banda/{id}/edit</code>	<code>app_banda_edit</code>	Editar datos de la banda (admin)

Método HTTP	URL	Nombre de ruta	Descripción
GET	/banda/{id}/solicitar-union	app_banda_form_solicitar_union	Formulario de solicitud de unión
POST	/banda/{id}/solicitar-union	app_banda_solicitar_union	Enviar solicitud de unión
GET	/banda/{id}/solicitudes	app_banda_solicitudes	Panel de gestión de la banda (admin)
POST	/banda/{id}/salir	app_banda_salir	Salir voluntariamente de la banda
POST	/banda/solicitud/{id}/aceptar	app_banda_aceptar_solicitud	Aceptar solicitud pendiente (admin)
POST	/banda/solicitud/{id}/rechazar	app_banda_rechazar_solicitud	Rechazar solicitud pendiente (admin)
POST	/banda/miembro/{id}/hacer-admin	app_banda_hacer_admin	Otorgar rol admin a un miembro
POST	/banda/miembro/{id}/quitar-admin	app_banda_quitar_admin	Revocar rol admin a un miembro
POST	/banda/miembro/{id}/expulsar	app_banda_expulsar_miembro	Expulsar miembro de la banda (admin)
POST	/banda/miembro/{id}/editar-rol	app_banda_editar_rol	Editar rol/instrumento de un miembro
POST	/banda/{banda}/invitar/{musico}	app_banda_invitar	Invitar músico a la banda (admin)
POST	/banda/invitacion/{id}/aceptar	app_banda_aceptar_invitacion	Músico acepta invitación

Método HTTP	URL	Nombre de ruta	Descripción
POST	/banda/invitacion/{id}/rechazar	app_banda_rechazar_invitacion	Músico rechaza invitación
POST	/banda/{id}	app_banda_delete	Eliminar banda (admin)

12. Controladores

BandaController

Archivo: `src/Controller/BandaController.php`

Extiende `AbstractController` de Symfony y agrupa toda la lógica de negocio del módulo. Los puntos clave de diseño son:

Inyección de dependencias — Las dependencias (`EntityManagerInterface`, `BandaRepository`) se inyectan por parámetro en cada método de acción, siguiendo el patrón estándar de Symfony.

Resolución automática de entidades — Symfony resuelve automáticamente parámetros como `Banda $banda` o `MiembroBanda $miembro` directamente desde el `{id}` de la URL usando el `ParamConverter`. No hay necesidad de buscar manualmente en el repositorio.

Control de acceso por capas:

1. `$this->denyAccessUnlessGranted('IS_AUTHENTICATED_FULLY')` — Verifica que hay sesión activa.
2. `$this->getUser()->getMusico()` — Verifica que el usuario tiene perfil de músico.
3. `$this->esAdminDeBanda(...)` — Verifica que el músico es administrador de esa banda concreta.

MusicoController (modificado)

Archivo: `src/Controller/MusicoController.php`

El método `show()` se ha actualizado para calcular y pasar a la plantilla dos variables nuevas:

```
// Bandas que el VISITANTE administra y en las que el músico del perfil
// no tiene ya una relación activa (para mostrar botones de invitación)
$bandasParaInvitar = [...];

// Invitaciones pendientes que el músico del perfil tiene recibidas
// (solo visibles para el propio músico)
$invitaciones = [...];
```

13. Formularios y validaciones

BandaType

Archivo: `src/Form/BandaType.php`

El formulario de banda define los tipos de campo correctos para cada dato y añade restricciones de validación del componente Symfony Validator:

```
->add('nombre', TextType::class, [
    'constraints' => [
        new NotBlank(message: 'El nombre no puede estar vacío'),
        new Length(min: 2, max: 100),
    ],
])
->add('biografia', TextareaType::class, [
    'constraints' => [
        new NotBlank(message: 'La biografía no puede estar vacía'),
    ],
])
->add('generos', TextType::class, [
    'required' => false,    // Campo opcional
])
->add('anyo_formacion', IntegerType::class, [
    'constraints' => [
        new NotBlank(message: 'El año no puede estar vacío'),
        new Range(min: 1900, max: 2100),
    ],
])
->add('ubicacion', TextType::class, [
    'constraints' => [
        new NotBlank(message: 'La ubicación no puede estar vacía'),
        new Length(min: 3),
    ],
])
->add('latitud', HiddenType::class, ['required' => false])
->add('longitud', HiddenType::class, ['required' => false])
->add('imagen_url', FileType::class, [
    'mapped' => false,    // No mapea directamente a la entidad
    'required' => false,
    'constraints' => [
        new File(maxSize: '2M', mimeTypes: ['image/jpeg', 'image/png',
'image/webp'])
    ],
])
```

La opción `'mapped' => false` en `imagen_url` indica a Symfony que no intente asignar automáticamente el archivo subido al campo de la entidad. El controlador gestiona manualmente el movimiento del archivo al directorio de uploads y guarda solo el nombre del archivo en la entidad.

14. Vistas Twig

`templates/banda/show.html.twig` — Perfil público de la banda

Esta es la vista principal de una banda. Muestra:

- Imagen de portada (o icono SVG si no hay imagen).
- Nombre, ubicación, año de formación y géneros musicales.
- Biografía de la banda.
- Lista de miembros aceptados, con sus instrumentos mostrados como tags visuales y una estrella (★) para los administradores.

La vista recibe tres variables del controlador:

- `banda` — La entidad banda.
- `miembro_actual` — El `MiembroBanda` del usuario actual en esta banda (o `null` si no pertenece).
- `es_admin` — Booleano que indica si el usuario actual es admin.

Los botones se muestran condicionalmente:

```
{% if es_admin %}
    {# Editar banda + Gestionar banda #}
{% elseif miembro_actual is null and is_granted('IS_AUTHENTICATED_FULLY') %}
    {# Solicitar unirse #}
{% endif %}

{# Salir de la banda: solo para miembros aceptados que no sean el último admin #}
{% if miembro_actual is not null and miembro_actual.estado == 'aceptado' %}
    {% set total_admins_show = banda.miembroBandas|filter(mb => ...)|length %}
    {% if not es_admin or total_admins_show > 1 %}
        {# Botón salir #}
    {% endif %}
{% endif %}
```

`templates/banda/solicitudes.html.twig` — Panel de gestión (admin)

Vista exclusiva para administradores, accesible desde "Gestionar banda". Tiene dos secciones:

Sección 1 — Solicitudes pendientes: Lista todas las solicitudes con `estado='pendiente'`. Para cada una muestra el nombre del músico (enlazado a su perfil), su ubicación y los instrumentos con los que se presenta (`rolBanda`). El admin puede aceptar o rechazar.

Sección 2 — Miembros actuales: Lista todos los miembros con `estado='aceptado'`. Para cada uno:

- Badge "★ Admin" si tiene privilegios de administrador.
- Botón "Hacer admin" o "Quitar admin" (solo si hay más de un admin).
- Botón "Expulsar" (con confirmación JavaScript) — deshabilitado si es el único admin.
- Elemento `<details>` desplegable para editar el rol/instrumentos del miembro sin salir de la página.

`templates/banda/solicitar_union.html.twig` — Formulario de solicitud

Vista que muestra los instrumentos del músico solicitante como checkboxes. El músico selecciona los instrumentos con los que quiere participar en la banda. Si no tiene instrumentos en su perfil, el botón de envío aparece deshabilitado y se le invita a añadirlos primero.

templates/banda/index.html.twig — Listado de bandas

Vista de tarjetas con todas las bandas. Se corrigió un bug anterior donde se mostraba `banda.ubicacion` como título de la tarjeta en lugar de `banda.nombre`.

templates/musico/show.html.twig — Perfil del músico (modificado)

Se han añadido dos secciones nuevas:

Sección Bandas: Muestra las bandas a las que pertenece el músico como tags clicables. Si es administrador de una banda, aparece una ★ junto al nombre. Usa `musico.miembroBandasAceptadas` para filtrar solo las bandas activas.

Sección Invitaciones pendientes: Solo visible para el propio músico (comprobando `app.user == musico.usuario`). Muestra las bandas que le han invitado con botones de Aceptar / Rechazar, cada uno con su propio formulario POST protegido con CSRF.

15. Navbar dinámico

Archivo: `templates/base.html.twig`

Se ha añadido un menú desplegable "Gestiona tu banda" a la barra de navegación, visible únicamente para usuarios autenticados que tengan un perfil de músico. El dropdown muestra:

1. Siempre: enlace "+ **Crear banda**".
2. Si el músico pertenece a alguna banda: separador y hasta 3 bandas, cada una con una insignia " (Admin)" si tiene ese rol.

```
{% if app.user.musico is not null %}
<div class="nav-item dropdown">
  <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown">
    Gestiona tu banda
  </a>
  <ul class="dropdown-menu dropdown-menu-dark dropdown-menu-end">
    <li><a class="dropdown-item" href="{{ path('app_banda_new') }}">+ Crear
banda</a></li>
    {% set mis_memberships = app.user.musico.miembroBandasAceptadas %}
    {% if mis_memberships|length > 0 %}
      <li><hr class="dropdown-divider"></li>
      {% for mb in mis_memberships|slice(0, 3) %}
        <li>
          <a class="dropdown-item" href="{{ path('app_banda_show', {id:
mb.banda.id}) }}">
            {{ mb.banda.nombre }}
            {% if mb.esAdministrador %}<small class="text-muted">
(Admin)</small>{% endif %}
          </a>
        </li>
      {% endfor %}
    {% endif %}
  </ul>
{% endif %}
```

```
</div>
{% endif %}
```

El filtro `|slice(0, 3)` limita la lista a 3 bandas máximo para no sobrecargar el menú. Bootstrap 5 gestiona la apertura/cierre del dropdown mediante JavaScript.

16. Seguridad y protección CSRF

Protección CSRF (Cross-Site Request Forgery)

Todas las acciones que modifican datos (cualquier petición POST) están protegidas con tokens CSRF. Symfony genera un token único por formulario/acción que se incluye como campo oculto y se valida en el controlador antes de ejecutar cualquier cambio.

Ejemplo en la plantilla:

```
<form method="post" action="{{ path('app_banda_aceptar_solicitud', {id:
miembro.id}) }}">
    <input type="hidden" name="_token" value="{{ csrf_token('aceptar_solicitud_' ~
miembro.id) }}">
    <button type="submit">Aceptar</button>
</form>
```

Validación en el controlador:

```
if ($this->isCsrfTokenValid('aceptar_solicitud_' . $miembro->getId(), $request-
>request->get('_token'))) {
    // Solo aquí se ejecuta la acción
    $miembro->setEstado('aceptado');
    $entityManager->flush();
}
```

Si el token no es válido, la acción simplemente no se ejecuta (no se lanza un error, para no revelar información de seguridad).

Control de acceso

El control de acceso se aplica en tres niveles:

1. **Autenticación** — `$this->denyAccessUnlessGranted('IS_AUTHENTICATED_FULLY')` bloquea todas las acciones sensibles para usuarios no autenticados.
2. **Perfil de músico** — Se comprueba que el usuario tiene un **Musico** asociado. Sin perfil de músico no se puede crear ni gestionar bandas.
3. **Administrador de banda** — El método privado `esAdminDeBanda()` se usa para proteger todas las acciones administrativas (aceptar solicitudes, expulsar, editar, borrar la banda...).

17. Migraciones de base de datos

Para aplicar todas las migraciones en el entorno Docker, dentro del contenedor:

```
# Entrar al contenedor (desde PowerShell en Windows)
docker exec -it <nombre-contenedor-php> bash

# Limpiar caché (si hay errores de caché de Symfony)
rm -rf var/cache/ && php bin/console cache:warmup

# Ejecutar migraciones pendientes
php bin/console doctrine:migrations:migrate
```

Si al ejecutar las migraciones aparece un aviso sobre migraciones en la base de datos que no están en el proyecto (de otras ramas), es seguro responder "yes": Symfony las marcará como ejecutadas sin volver a ejecutarlas.

Revertir migraciones

Cada migración tiene un método `down()` que deshace los cambios:

```
# Revertir la última migración aplicada
php bin/console doctrine:migrations:execute --down DoctrineMigrations\\VersionXXXX
```

Resumen de archivos modificados

Archivo	Tipo de cambio
src/Entity/Banda.php	Añadidos campos <code>nombre</code> , <code>latitud</code> , <code>longitud</code>
src/Entity/MiembroBanda.php	Añadidos campos <code>estado</code> , <code>es_administrador</code> ; <code>rol_banda</code> nullable
src/Entity/Musico.php	ManyToMany instrumentos, <code>getInstrumentos()</code> , <code>getMiembroBandasAceptadas()</code>
src/Entity/InstrumentoSistema.php	Nueva entidad (de rama instrumentos)
src/Entity/InstrumentoPersonalizado.php	Nueva entidad (de rama instrumentos)
src/Form/BandaType.php	Tipos de campo correctos + validaciones NotBlank/Length/Range
src/Controller/BandaController.php	Implementación completa de 18 rutas
src/Controller/MusicoController.php	Método <code>show()</code> ampliado con invitaciones y bandas para invitar

Archivo	Tipo de cambio
migrations/Version20260527120000.php	Nombre banda + estado/es_administrador en miembro_banda
migrations/Version20260527130000.php	Latitud y longitud en banda
templates/base.html.twig	Dropdown "Gestiona tu banda" en navbar
templates/banda/_form.html.twig	Campo nombre + integración Google Places
templates/banda/show.html.twig	Perfil completo con miembros, instrumentos, botones condicionales
templates/banda/index.html.twig	Corrección: mostrar <code>banda.nombre</code> en lugar de <code>banda.ubicacion</code>
templates/banda/solicitudes.html.twig	Panel completo: pendientes, miembros, expulsar, editar rol
templates/banda/solicitar_union.html.twig	Nuevo: formulario con instrumentos del músico como checkboxes
templates/musico/show.html.twig	Sección bandas del músico + sección invitaciones pendientes
config/services.yaml	Parámetros para directorio uploads/bandas y clave Google Places